FPGA-based Reservoir Computing with On-chip Learning for MIMO Channel Prediction

Chunxiao Lin, Ummay Sumaya Khan, Meiyu Zheng, Muhammad Farhan Azmine, Lingjia Liu, *IEEE Fellow*, Daniel J. Jakubisin, *Member, IEEE*, and Yang Yi, *Senior Member, IEEE*

***Abstract*—The Echo State Network (ESN) is a lightweight recurrent neural network model well-suited for real-time temporal data analysis. Its fast convergence and low training data requirements make it a promising candidate for Next-Generation (Next-G) wireless applications such as Multiple-Input Multiple-Output (MIMO) channel prediction, where stringent in-subframe latency and reliability are required. However, current ESN solutions for Next-G tasks are primarily implemented on CPUs or GPUs, with some exploration of FPGA acceleration for ESN inference. These approaches fail to fully exploit the inherent hardware-friendly nature of ESNs, particularly in the learning phase, which remains the most time- and energy-intensive component in real-time training and prediction. In this paper, we propose a complete on-chip ESN architecture featuring a recursive least squares learning module implemented on FPGA. Several architectural optimizations are introduced for both inference and training to enhance throughput and resource utilization efficiency, while explicitly addressing the challenges posed by the high-dimensional, real-time nature of MIMO channel prediction. Experimental results demonstrate that the proposed on-chip ESN achieves a high processing speed and superior energy efficiency compared with CPU- and GPU-based channel predictor implementations. Furthermore, the ESN-based channel predictor outperforms traditional autoregressive and other machine-learning approaches in normalized mean squared error (NMSE), confirming its effectiveness for fast and accurate MIMO channel prediction.**

***Index Terms*—FPGA, Reservoir computing, Echo state network, On-chip learning, Channel prediction, Machine learning**

I. Introduction

THE growing computational and energy demands of modern AI models pose significant challenges for portable, resource-constrained devices. This limitation is particularly critical in the context of modern Next-Generation (Next-G) communication, Internet of Things, and autonomous vehicles, where real-time processing, low-latency decision-making, and high energy efficiency are essential. Although recent studies have explored edge-intelligence solutions for IoT and vehicular networks [1], [2], they typically rely on complex reinforcement-learning methods and require dedicated edge-server infrastructure, which may not fit scenarios that demand lightweight, low-latency on-device intelligence. In this landscape, Echo State Networks (ESNs), a type of lightweight Recurrent Neural Networks (RNNs) inspired by neuroscience, emerge as a promising solution. ESNs provide simplified training process with reduced number of weights that need to be learned to perform complex tasks with limited training samples and computational resources [3]. Unlike conventional RNNs, where all weights are trained, ESNs employ a fixed, randomly initialized internal reservoir with only a trainable linear readout layer [4]. This architecture enables fast convergence and low computational overhead, making ESNs well-suited for low-power and low-latency applications. In the Multiple-Input Multiple-Output (MIMO) channel prediction task, extreme channel dynamics requires AI models to perform real-time online training and inference, demanding a sub-millisecond processing latency. ESN-based approaches are therefore considered strong candidates due to their lightweight and efficient training process, continuously learning from the latest channel observations and providing predicted channels to inform future transmission strategies [5].

However, the majority of ESN solutions for wireless channel prediction tasks are still based on CPUs or GPUs, which focus mainly on algorithm design and do not explore hardware optimizations in throughput and energy efficiency [6], [7]. As Next-G wireless systems increasingly emphasize edge intelligence, there is a growing need for low-latency and low-power ESN implementations. Field-Programmable Gate Arrays (FPGAs), as reconfigurable hardware platforms, present a promising alternative due to their ability to exploit parallelism at both the instruction and data levels while supporting rapid development cycles. Compared with conventional computing platforms such as GPUs and ASICs, FPGA-based implementations achieve a favorable balance between performance and flexibility, making them well-suited for accelerating ESN models. Among FPGA-based ESN studies, most efforts have focused on optimizing the reservoir design, such as creating simplified reservoir structures [8] or improving mathematical operations [9]–[12] to reduce computational overhead during ESN inference and minimize hardware resource usage. However, hardware realization of a complete end-to-end ESN system remains limited, primarily due to the challenges associated with online training.

In response to these challenges, this work presents a complete FPGA-based real-time ESN system that integrates high-speed inference with fully on-chip learning. Unlike traditional offline training, which requires collecting and buffering large batches of reservoir states and target output before

Efforts sponsored by the U.S. Government under the Training and Readiness Accelerator II (TReX II), OTA. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. *(Corresponding author: Yang Yi.)*

Chunxiao Lin, Ummay Sumaya Khan, Meiyu Zheng and Muhammad Farhan Azmine are with the Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, VA, USA. Email: {chunxiaol, ummaysumaya, meiyuzheng, muhammadfarhan}@vt.edu

Lingjia Liu is a professor of Wireless@Virginia Tech, the Bradley Department of Electrical and Computer Engineering, Virginia Tech.

Daniel J. Jakubisin is a research associate professor with the Hume Center for National Security and Technology, National Security Institute, Virginia Tech and a member of Wireless@VT.

Yang Yi is a professor in the Bradley Department of Electrical and Computer Engineering and the director of the BRain-Inspired Computing, Communications, and Security Lab (BRICCS @VT) at Virginia Tech.

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses. This is the author's accepted manuscript of the paper published in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, DOI: 10.1109/TCAD.2026.3715776.

computation, resulting in substantial latency, online training algorithms such as recursive least squares (RLS) [13] can update model parameters sample-by-sample. Building on this advantage, we implement on-chip learning using an FPGA-accelerated RLS architecture, enabling low-latency, real-time online ESN adaptation. The primary contributions of this work are summarized as follows:

1) To the best of our knowledge, this is the first FPGA hardware architecture for RLS-based on-chip ESN learning, with both inference and training integrated into a unified, fixed-point real-time system and validated on a hardware testbed for MIMO channel prediction.
2) We develop dedicated hardware for each layer of the system: a DSP-array-based MAC_DA unit with a range-addressable LUT(RALUT)-based activation for the reservoir layer, a systolic-array-based DSP_SA unit for the output layer, and a matrix-inversion-free RLS learning block that reuses these MAC units. Through parallelism and pipelining, the architecture enables real-time online learning on high-dimensional data, unlike prior onchip ESNs.
3) Comprehensive evaluations are conducted on the public Mackey–Glass benchmark and a realistic MIMO-OFDM channel prediction task, with further validation on an FPGA platform. It shows that the proposed accelerator outperforms non-ML and ML baselines in NMSE while delivering more than $1600\times$ and $290\times$ higher energy efficiency than CPU- and GPU-based implementations, respectively, highlighting its practicality for real-world applications.

II. BACKGROUND

A. Echo State Networks

A conventional ESN follows a three-stage architecture consisting of an input interface, a recurrent reservoir, and a linear readout layer [4]. The input layer contains N_i units, the reservoir consists of N_r recurrent units, and the output layer comprises N_o units. At discrete time index t, the corresponding input, reservoir state, and output vectors are denoted by $\mathbf{x}(t) = (x_1(t), \ldots, x_{N_i}(t))$, $\mathbf{r}(t) = (r_1(t), \ldots, r_{N_r}(t))$, and $\mathbf{y}(t) = (y_1(t), \ldots, y_{N_o}(t))$, respectively.

The ESN dynamics are characterized by three weight matrices: an input-to-reservoir projection matrix $\mathbf{W}_{in} \in \mathbb{R}^{N_r \times N_i}$, which projects the external input into the reservoir; a recurrent reservoir weight matrix $\mathbf{W}_r \in \mathbb{R}^{N_r \times N_r}$, which governs the internal reservoir dynamics; and an output readout matrix $\mathbf{W}_{out} \in \mathbb{R}^{N_o \times (N_r + N_i)}$, which maps the reservoir state to the output space. The reservoir state evolves according to the following nonlinear update equation

$$\mathbf{r}(t+1) = \phi(\mathbf{W}_{in}\mathbf{x}(t+1) + \mathbf{W}_r\mathbf{r}(t)), \tag{1}$$

where $\phi(\cdot)$ denotes a nonlinear activation function applied element-wise to introduce nonlinear temporal dynamics.

The output is subsequently generated through a linear readout applied to the concatenation of the current reservoir state and the input vector, given by

$$\mathbf{y}(t+1) = \mathbf{W}_{out}\begin{bmatrix}\mathbf{r}(t+1)\\ \mathbf{x}(t+1)\end{bmatrix}. \tag{2}$$

While classical ESN models often employ large reservoirs with tens to thousands of neurons to enrich system dynamics, recent hardware-oriented and application-specific designs have shown that small reservoir sizes can show good results in wireless tasks with fast-converging supervised learning algorithms. The input and output layer dimensions are typically determined by the application-specific feature representation and prediction targets.

The training of ESNs can be conducted in two primary ways, **offline** and **online**, both of which determine the optimal output weight matrix $\mathbf{W}_{out}$ while keeping $\mathbf{W}_{in}$ and $\mathbf{W}_r$ fixed. **Offline training** computes $\mathbf{W}_{out}$ after all training samples have been processed, using the complete set of reservoir states and target outputs, typically via linear regression with Tikhonov regularization. While this allows thorough optimization, it requires storing and processing large datasets, introducing latency that makes it less suitable for real-time applications.

In contrast, **online training** adjusts the output weights in real-time, as the input sequence is being processed. Algorithms such as RLS are used to update $\mathbf{W}_{out}$ incrementally, enabling the ESN to dynamically adapt to changes in the input data. Online training is particularly advantageous in real-time applications where immediate responses are crucial, though it may involve greater computational complexity due to the need for continuous updates.

B. Online Training Methods for ESN

In this section, we discuss a collection of available algorithms for training of output weights W_{out} of the ESN as well as their advantages and disadvantages.

Stochastic Gradient Descent (SGD) Algorithm: SGD is one of the foundational online learning methods widely used in training reservoir readout layers. It uses differentiation of the quadratic error function $\mathbf{e}_{\text{sgd}}(t)$ at a time-step t as gradient for updating the weights of ESN incrementally [14]. For the linear readout with weights $\mathbf{W}_{\text{sgd}}$ and the extended state $\mathbf{z}(\mathbf{t}) = [\mathbf{r}(\mathbf{t}); \mathbf{x}(\mathbf{t})]$ that concatenates the input vector and the state vector, the update takes the following equation:

$$\mathbf{W}_{\text{sgd}}(t) = \mathbf{W}_{\text{sgd}}(t-1) - \alpha_{\text{sgd}} \cdot \mathbf{e}_{\text{sgd}}(t)\mathbf{z}^{\mathbf{T}}(\mathbf{t}), \tag{3}$$

where α_{sgd} is the learning rate which is tuned to be small to avoid instability in the convergence process.

Despite the low complexity, its limitations include slower convergence compared to RLS, along with being sensitive to the proper tuning of the learning rate, which might lead the ESN to achieve sub-optimal performance.

Reward-modulated Hebbian Algorithm: Reward-modulated Hebbian (RMH) learning is another online approach explored for training ESN output weights, particularly useful when an analytic error function is hard to obtain or target values are unavailable [14]. Following the rule of Legenstein et al. [15], it updates the weights through a Hebbian term modulated by a reward signal, where the modulation is driven by the approximated noise of the network output and the deviation of the instantaneous reward from its recent-average baseline. However, without prior

knowledge of the system noise or target values, RMH was found to be less precise and slower to converge [14].

Recursive Least Squares Algorithm: In contrast to SGD and RMH—both of which suffer from slow or imprecise convergence—the RLS algorithm [16] provides a more accurate and rapidly adapting solution for online ESN training. By performing second-order updates that incorporate information from all past samples, RLS enables the model to adjust its weights efficiently and robustly as new data arrives.

For real-time wireless applications, where channel conditions evolve quickly and data streams in continuously, the sample-by-sample update property of RLS allows the ESN to track dynamic system behavior with significantly faster convergence and higher prediction accuracy. The optimal RLS weight update at training timestep t is formulated as follows:

$$\mathbf{W}_{\text{out}}(t) = \arg\min_{\mathbf{W}_{\text{out}}} \sum_{m=1}^{t} \lambda^{t-m} \|\mathbf{y}(m) - \mathbf{W}_{\text{out}}\mathbf{z}(m)\|_2^2 \quad (4)$$

where $\lambda \in (0, 1]$ is the forgetting factor. For RLS optimization, the forgetting factor should be less than 1 to give more importance to error minimization related to recent sample observations compared to the past. This makes the algorithm adaptive to an environment, which is crucial for wireless applications where the system dynamics can change gradually with time. In general, RLS maintains a covariance matrix $\mathbf{\Phi}$ that summarizes the second-order statistics of all past inputs under exponential forgetting, and at each new sample uses this accumulated information to determine both the magnitude and direction of the weight update, avoiding any explicit matrix inversion. A step-by-step process through which the RLS updates the weights sample by sample is provided below:

1) **Initialization:** The covariance matrix Φ and W_{out} are initialized at t= 0. Here, $\Phi(0)$ is initialized as an identity matrix and $W_{out}(0)$ is a zero matrix.
2) **Compute Kalman gain vector:** The gain vector $k(t)$, used to adjust the output weights for the sample collected at timestep t, is computed as:

$$\mathbf{k}(t) = \frac{\mathbf{\Phi}(t)\mathbf{z}(t)}{\lambda + \mathbf{z}^{\mathbf{T}}(t)\mathbf{\Phi}(t)\mathbf{z}(t)} \quad (5)$$

Here, $\mathbf{z}(\mathbf{t}) = [\mathbf{r}(\mathbf{t}); \mathbf{x}(\mathbf{t})]$ is the extended state.
3) **Compute the Error:** The W_{out} at timestep t-1 is used to find the error between target and predicted output at t timestep, which quantifies the new information carried by the current sample relative to the existing model:

$$\mathbf{e}_{\text{rls}}(t) = \mathbf{y}(t) - \mathbf{W}_{\text{out}}(t-1)\mathbf{z}(t) \quad (6)$$

4) **Update the output weights:** The prediction error $\mathbf{e}_{\text{rls}}(t)$ is then used to correct $\mathbf{W}_{out}$ along the direction of $\mathbf{k}(t)$, scaled by the residual magnitude:

$$\mathbf{W}_{\text{out}}(t) = \mathbf{W}_{\text{out}}(t-1) + \mathbf{e}_{\text{rls}}(t)\mathbf{k}^{\mathbf{T}}(t) \quad (7)$$

5) **Update the covariance matrix:** The covariance matrix is then recursively updated to absorb the new sample's contribution, while the factor λ^{-1} keeps it responsive to non-stationary dynamics:

$$\mathbf{\Phi}(t+1) = \lambda^{-1}\left(\mathbf{\Phi}(t) - \mathbf{k}(t)\left[\mathbf{z}^{\mathbf{T}}(t)\mathbf{\Phi}(t)\right]\right), \quad (8)$$

which is then reused from step 2 for the next sample.

Although the recursive covariance update adds computational complexity, this can be mitigated through hardware-oriented optimizations. Moreover, while RLS requires supervised target information, this assumption is naturally satisfied in MIMO channel prediction through pilot-based channel estimates. As a result, RLS is adopted in this work to enable accurate and low-latency on-chip ESN training.

Time Complexity: Let $L = N_i + N_r$. For inference, the reservoir update and the linear readout together result in a complexity of $\mathcal{O}((N_r + N_o)L)$. For the RLS training, each per-sample update is dominated by the covariance-related operations in Eqs. 5 and 8, each of $\mathcal{O}(L^2)$, leading to a per-sample training complexity of $\mathcal{O}\big(L^2 + N_o L\big)$.

III. State-of-the-art research

On-chip supervised learning techniques for ESNs have been explored in prior work; however, research in this area remains relatively limited. In [17], a digital ESN system was implemented using ridge regression and applied to MIMO communication tasks, which only processes binary inputs and lacks detailed architecture of the training block. Antonik, instead, implemented the gradient descent algorithm in [18]. While simpler in its conceptual approach, this method introduces higher computational complexity due to the iterative nature of weight updates. More recently, Galán-Prado et al. [19] proposed a hardware design for noniterative supervised learning on ESN, leveraging max-plus algebra to implement multidimensional pseudo regression. This approach significantly reduces hardware utilization at the cost of reduced prediction accuracy. To the best of our knowledge, no existing work has reported a hardware-accelerated on-chip ESN implementation employing the RLS learning algorithm. Moreover, existing implementations are generally limited in their ability to process high-dimensional data, which is impractical for Next-G wireless systems where the number of features grows rapidly with increasing radio system scale.

Although several FPGA-based RLS implementations have been proposed for specific applications—such as anti-jamming systems [20] and fixed-tap filter designs [21], these architectures are highly application-specific and not directly applicable to ESN training. RLS has also been applied to ESN simulations for wireless applications [13], yet hardware acceleration of this remains unexplored. In this work, we adopt an ESN-specific RLS formulation that avoids matrix inversion and develop a hardware-optimized architecture for MIMO channel prediction. While its parameterized datapath is reusable for workloads of comparable size, the design choices themselves—a compressed high-dimensional datapath, pipelined low latency, and pilot-supervised online RLS—are driven by the requirements of the channel prediction task.

IV. Design of On-chip ESN system

The proposed on-chip ESN system is designed with three primary components: the reservoir layer, the output layer, and the RLS training block. In the reservoir layer, we propose an optimized reservoir neuron architecture that integrates a

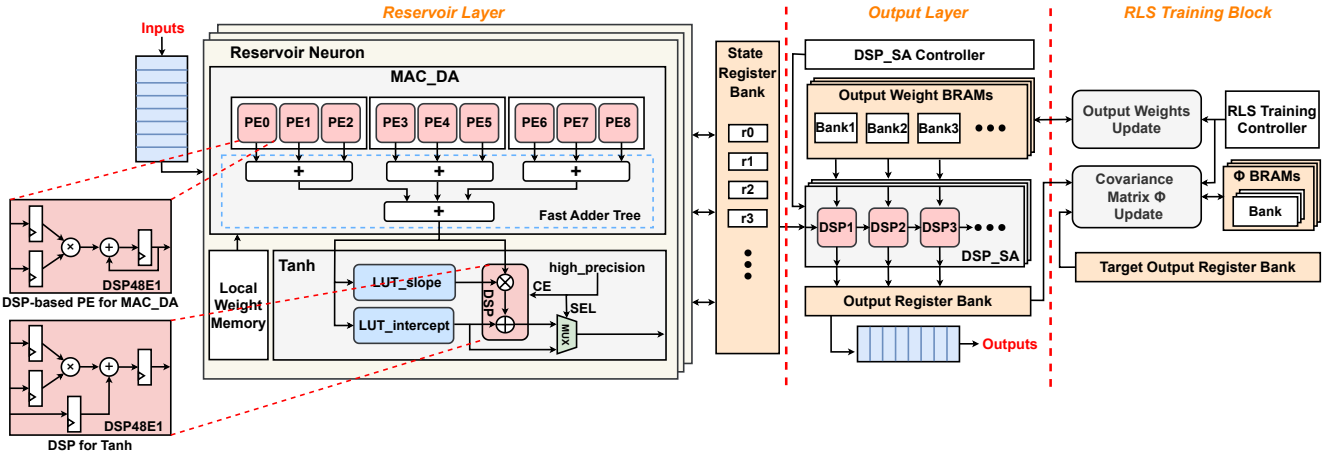


Fig. 1. Overall architecture of the on-chip ESN system. The details of the optimized reservoir neuron architecture are also shown.

MAC_DA unit and a range-addressable LUT (RALUT)-based hyperbolic tangent (tanh) block, enabling efficient state updating. For the output layer, we developed a DSP_SA unit that enables compressed operations on the output neurons by leveraging a systolic array structure, thereby achieving a balance between throughput and resource utilization. In the RLS training block, we implemented a hardware-optimized architecture for the RLS algorithm, integrating the MAC_DA and DSP_SA units to enhance computational efficiency. These three blocks are then integrated with a cohesive fixed-point design, providing a complete on-chip ESN system. An overview of the system architecture is illustrated in Figure 1.

A. Reservoir Layer Design

In each neuron of the reservoir layer, the state updating process is performed as outlined in Equation 1, which consists of a linear combination of the current input $\mathbf{x}(t+1)$ and the previous reservoir state $\mathbf{r}(t)$, followed by a nonlinear function applied to the resulting Sum of Product (SoP). Through mathematical reformulation, the original linear combination

$$\mathbf{w}_{\text{in},i} \cdot \mathbf{x}(t+1) + \mathbf{w}_{\text{r},i} \cdot \mathbf{r}(t) \tag{9}$$

can be expressed as a single multiply–accumulate (MAC) operation by concatenating the input and reservoir vectors along with their corresponding weights:

$$\{\mathbf{w}_{\text{in},i}; \mathbf{w}_{\text{r},i}\} \cdot \{\mathbf{x}(t+1); \mathbf{r}(t)\}. \tag{10}$$

Here, $\mathbf{w}_{\text{in},i}$ and $\mathbf{w}_{\text{r},i}$ denote the i-th rows of the input and reservoir weight matrices $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{r}}$, respectively.

Based on this reformulation, a cost-efficient reservoir neuron architecture is proposed. Each neuron integrates a MAC_DA unit for vector multiplication and a RALUT-based tanh activation function. By jointly leveraging DSP48E1 slices and Configurable Logic Blocks (CLBs), the proposed design can achieve high throughput with low CLB utilization.

As illustrated in Figure 1, each MAC_DA unit leverages nine DSPs to execute nine parallel MAC operations. In the first stage, this operation is iteratively performed until all elements of the input vector are processed. The resulting nine partial SoPs are then passed through a two-stage high-speed adder tree to generate the final SoP. To efficiently compress the partial results, 3-input carry-save adders (CSAs) are employed together with a final carry-lookahead adder (CLA) layer, where four CSA–CLA blocks are used to complete the accumulation.

In this way, the proposed MAC_DA unit takes $\left\lceil \frac{N_i+N_r}{9} \right\rceil + 2$ clock cycles to complete the full MAC operation and forward the final SoP to the activation function. Since the vector multiplication is performed independently within each neuron, the resource utilization scales with N_r, making the MAC_DA unit particularly cost-efficient for small reservoirs.

The tanh function is implemented by a LUT-based approximation with linear interpolation, together with a range-addressable LUT optimization.

Here, the SoP output from the MAC_DA unit is represented as a fixed-point value s consisting of a sign bit, an integer part, and a fractional part. Considering the centrosymmetric property of the tanh function, the computation is performed only on the absolute value of s, with a sign inversion applied to the final output when s is negative. Furthermore, since $\tanh(s) \approx 1$ for $s > 8$ with an error below 10^{-6}, the effective input range of the LUT is truncated to $[0, 8]$.

To approximate the tanh function within this range, a lookup table named LUT_intercept stores precomputed tanh values at 2^a uniformly sampled points, where the address width a is set to 8 in our implementation. A sample index s_c derived from $|s|$ selects the nearest stored value, and for inputs that fall between sampled points, linear interpolation is employed to improve accuracy. An additional LUT, referred to as LUT_slope, stores the local slope of the tanh function at each sampled point using the same index. The interpolated tanh output is then computed as

$$\tanh(s) \approx \text{slope}_s \times \Delta_s + \text{intercept}_s, \tag{11}$$

where Δ_s denotes the offset between the input value and its nearest sample. As shown in Figure 1, an additional DSP is adopted for this operation.

To further reduce dynamic power consumption, a range-addressable LUT optimization is introduced. Since the slope of $\tanh(s)$ decreases as s increases, the tanh output varies little in the large-s region. Accordingly, the input range $[0, 8]$ is partitioned into a high-precision region $[0, t)$ and a low-precision region $[t, 8]$, where t is a predefined threshold. For

inputs $s < t$, the DSP-based linear interpolation is enabled and its output is selected as the tanh result, whereas for $s \geq t$, the interpolation is bypassed and the value stored in LUT_intercept is used directly. This selective computation significantly reduces dynamic power consumption in the tanh block while only introducing negligible approximation error.

B. Output Layer Design

For the output layer design, predicting a large number of outputs presents a challenge to resource efficiency. Applying a separate MAC_DA unit for each output neuron can lead to excessive hardware utilization in the high-dimensional dataset. To address this, we developed a systolic-array-based processing unit, termed DSP_SA, to perform compressed operations across multiple output neurons. This architecture takes advantage of the systolic array architecture to enhance throughput, ensuring efficient processing even with high-dimensional data, enabling the on-chip ESN for MIMO channel prediction. Figure 2(a) shows the architecture of the ESN output layer utilizing the DSP_SA unit, which is a DSP-based 1D systolic array for the matrix-vector multiplication (MVM) operation.

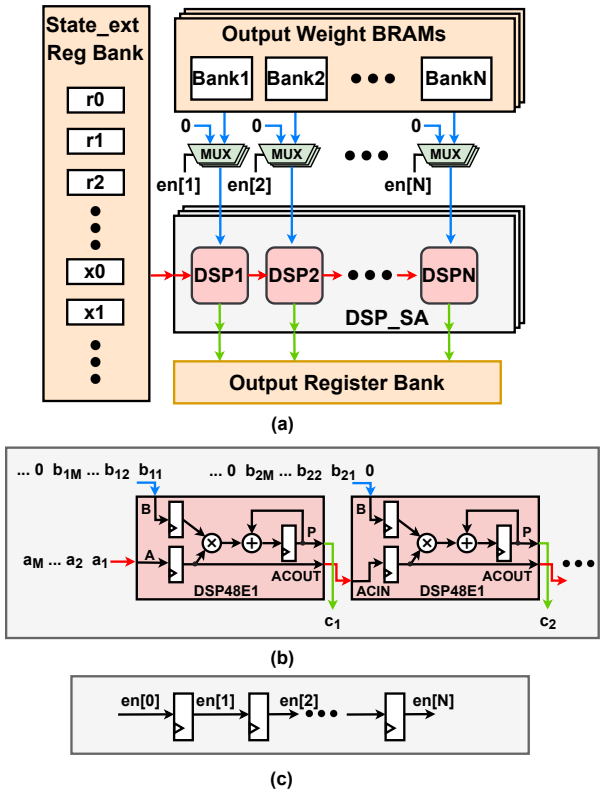


Fig. 2. Architecture of (a) the proposed output layer, (b) the DSP_SA unit, and the control signal generator.

1) DSP_SA Unit for matrix-vector multiplication: Figure 2(b) illustrates our proposed DSP_SA unit, which efficiently handles the MVM operation in both the ESN inference and the RLS learning process. The design leverages the DSP48E1 slices in 7-series FPGAs, whose input–output cascading enables both MAC execution and seamless data propagation within the systolic array.

For the initial DSP and the subsequent DSPs in the DSP_SA unit, one of the inputs for the MAC operation can be selected from either the A port or the ACIN port, which receives vector elements from the memory or the previous DSP. The B port serves as the other input for the current DSP, receiving elements from a specific row of the input matrix. The inputs from the A/ACIN and B ports are multiplied using the built-in multiplier, and the resulting product is accumulated with the previous output P to generate the new output P. Simultaneously, the input from A/ACIN is registered and then cascaded to the next DSP in the array through the ACOUT port.

By combining N customized DSPs into a 1D systolic array, we can perform any multiplication between a matrix of size (N, M) and a vector of size $(M, 1)$, where M can be any positive integer. Assume the vector is $\mathbf{a} = [a_1, a_2, \ldots, a_M]^T$, and the matrix is

$$\mathbf{b} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1M} \\ b_{21} & b_{22} & \cdots & b_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ b_{N1} & b_{N2} & \cdots & b_{NM} \end{bmatrix},$$

the resulting vector will be $\mathbf{c} = [c_1, c_2, \ldots, c_N]^T$, where $c_i = \sum_{k=1}^{M}(b_{ik} \cdot a_k)$.

Following the systolic timing, DSP_1 begins accumulating $\sum_k b_{1k}a_k$ from the first cycle, while each subsequent DSP_i starts one cycle later as the vector element cascades in from DSP_{i-1}, with zeros fed to the B ports of the DSPs not yet active. After M such steps, each DSP_i holds the final sum-of-products (SoP) c_i of the output vector $\mathbf{c}$.

In this configuration, a DSP_SA with N DSPs performs the MVM operation with a latency of $(N + M + 1)$ clock cycles: the final element of the matrix input $\mathbf{b}$ reaches the last DSP at the $(N + M - 2)$ clock edge, and an additional 3 cycles are needed to produce the final SoP. Since the MVM involves M multiplications and $(M - 1)$ additions for each of the N rows, the overall computational efficiency of the DSP_SA is

$$\frac{(M \cdot N \text{ multiplications} + (M \cdot N - N) \text{ additions})}{(N + M + 1)} \text{ per cycle.}$$

In practical cases such as our channel prediction task where $N \ll M$, this approximates to about (N multiplications + N additions) per clock cycle, indicating that all DSPs operate at full capacity for high circuit utilization and efficiency.

2) DSP_SA-based Output Layer: With the proposed DSP_SA unit, we can implement the high-density output layer using a systolic array-based compressed architecture.

As shown in Equation 2, the operation across the entire output layer can be expressed as a multiplication between the output weight matrix $\mathbf{W}_{out}$ of dimension $(N_o, (N_i + N_r))$ and the extended state vector $\mathbf{r_e x} = \{\mathbf{r}; \mathbf{x}\}$ with dimension $((N_i + N_r), 1)$, resulting in an output vector $\mathbf{y}$ of dimensions $(N_o, 1)$. Given that N_i and N_o are relatively large in our application, we divide $\mathbf{W}_{out}$ into N_o/N sub-matrices, each with dimensions $(N, (N_i + N_r))$. Correspondingly, N_o/N DSP_SA units, each with a depth of N, are deployed to perform the multiplication between each submatrix and the vector $\mathbf{r_e x}$, generating N SoPs in each DSP_SA unit.

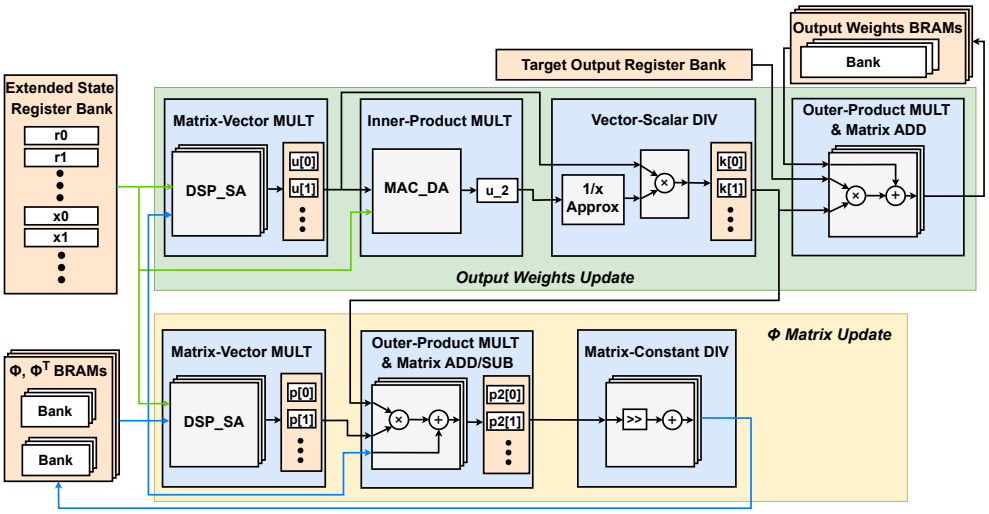


Fig. 3. Block Design of the RLS Training.

To accommodate the high-dimensional data, the large output weight matrix $\mathbf{W}_{\text{out}}$ is partitioned and stored across N_o BRAMs, each with a depth of N_i+N_r, containing the weights required for the corresponding DSP in the DSP_SA unit. A shift register of length N is implemented to generate N enable signals $en[1]-en[N]$ with consecutive delays for all N BRAMs within a single DSP_SA unit, as illustrated in Figure 2(c). These control signals can be shared across all DSP_SA units, as the units operate in parallel. Additionally, a $\log_2(N_i+N_r)$-bit counter is used to generate the read addresses for the BRAMs. This mechanism ensures that each DSP receives either the correct weight value or a zero, depending on the active select signal, allowing for precise and efficient data flow during matrix-vector multiplication.

In the output layer design, a total of N_o/N DSP_SA units are required for the given ESN structure. Since all DSP_SAs operate in parallel, the overall latency remains at $(N+M+1)$ clock cycles, which corresponds to $(N+N_i+N_r+1)$ in this case. This directly guides the selection of the number of DSPs within each DSP_SA unit, since decreasing the number of DSPs per unit can reduce the latency of the output layer. Considering that the target application emphasizes high throughput, we select eight DSPs per DSP_SA unit to achieve low latency. This choice introduces only a small fanout overhead from the state vector register, as only one state value is broadcast to all DSP_SA units at one time.

Finally, all the SoPs generated are stored in the output registers, making them available for the ESN inference readout and the subsequent RLS training process.

C. RLS Training Block Design

To enable fully on-chip learning for the ESN, we designed a hardware-optimized RLS architecture for updating the ESN output weights, which, as detailed in Section II-B, implements recursive operations without requiring matrix inversion. The RLS process is divided into two submodules, as depicted in Figure 1: the *output_weight_update* module, which computes the Kalman gain and updates the output weights, and the *covariance_matrix (Φ)_update* module, which updates the covariance matrix Φ. Since the high-dimensional MVM is the most time-consuming operation here, two DSP_SA units are adopted within the RLS block, one for each module, to optimize the processing time.

A detailed block diagram derived from Eqs. 5–8 is provided in Figure 3, where each equation is decomposed into the elementary hardware operations—MVM, inner/outer products, and scalar division—detailed in the two modules below.

1) Output_weight_update module: In this module, the initial operation is an MVM operation between the covariance matrix $\mathbf{\Phi}$ of dimension $((N_i+N_r),(N_i+N_r))$ and the extended state vector $\mathbf{r}_{\text{ex}}$ of dimension $((N_i+N_r),1)$. For this operation, we reuse the DSP_SA-based structure from the output layer, where the input matrix dimensions are adjusted from $(N_o,(N_i+N_r))$ to $((N_i+N_r),(N_i+N_r))$, while the vector dimension remains the same. The architecture is identical to the one in Figure 2, with fewer DSP_SAs being used. We denote the output vector of this operation as:

$$\mathbf{u}=\mathbf{\Phi}\cdot\mathbf{r}_{\text{ex}}. \tag{12}$$

Then we reuse the MAC_DA unit from the reservoir layer for the inner product calculation. The scalar generated from this operation is:

$$u_2=\mathbf{r}_{\text{ex}}^T\cdot\mathbf{u}. \tag{13}$$

In the third operation, we divide the vector $\mathbf{u}$ by the scalar u_2 to compute the Kalman Gain vector $\mathbf{k}$, using a LUT-based reciprocal multiplication. Taking the calculation of one element $k_i=\frac{u_i}{u_2}$ as an example, we approximate the operation in two steps:

$$u_3=reciprocal(u_2) \tag{14}$$

$$k_i=u_i\cdot u_3. \tag{15}$$

In hardware, we implemented the reciprocal unit shown in Figure 4 to perform equation 14, followed by a DSP-based multiplier for equation 15.

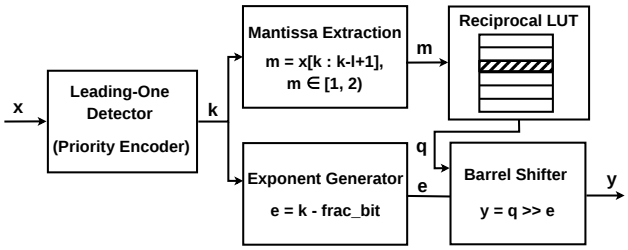


Fig. 4. Hardware implementation of the reciprocal unit

As illustrated in the figure, a priority encoder is first employed to detect the position of the most significant non-zero bit of the input x, referred to as the leading one and denoted by k. Once k is identified, the mantissa of x is extracted by selecting an l-bit segment m starting from the leading-one position, while the exponent e is computed by subtracting the number of fractional bits from k. This representation allows the input value to be expressed in a normalized form as

$$x = m \cdot 2^e. \tag{16}$$

Here, m is in the range of [1, 2), which is then sent to a small reciprocal LUT to get the reciprocal of m. Considering the MSB of m is fixed at 1, only 2^{l-1} locations for the LUT are needed. At the end, the reciprocal of x can be generated using a barrel shifter, with the following equation

$$reciprocal(x) = reciprocal_{LUT}(m) >> e \tag{17}$$

In our case, l is set to 5 with an NMSE of 10^{-4} compared to the numpy-based reciprocal function, which provides a high precision in our application.

The last two operations involve an outer-product multiplication and a matrix addition, which are used to compute ΔW and update the weight matrix W, respectively. Letting $\otimes$ denote the outer product, these operations are combined into a single computation composed of element-wise MAC operations, expressed as

$$W_{\text{new}} = \mathbf{e}_{\text{rls}} \otimes \mathbf{k} + W. \tag{18}$$

where $\mathbf{e}_{\text{rls}}$ is the prediction error defined in Equation 6 and $\mathbf{k}$ is the Kalman gain from Equation 15.

2) Covariance_matrix_update_module design: In the Φ*_matrix_update* module, the initial operation is a vector-matrix multiplication between the transpose of $\mathbf{r}_{\text{ex}}$ and the $\mathbf{\Phi}$ matrix, which shares the same inputs as Equation 12. If we denote the output vector of this operation as $\mathbf{p}$, it can be calculated as

$$\mathbf{p} = \mathbf{r}_{\text{ex}}^T \cdot \mathbf{\Phi}, \tag{19}$$

which can be reformulated as an MVM operation:

$$\mathbf{p} = (\mathbf{\Phi}^T \cdot \mathbf{r}_{\text{ex}})^T. \tag{20}$$

Given that the hardware implementation treats both row and column vectors (1xN and Nx1) equivalently in terms of representation, we can simply reuse the circuits from Equation 12 by sending the $\mathbf{\Phi}$ matrix in its transposed form. Considering that the $\mathbf{\Phi}$ matrix is saved in BRAMs, we use another set of BRAMs for the transposed matrix $\mathbf{\Phi^T}$.

The following two operations are an outer product multiplication and a matrix addition/subtraction, combined into a single element-wise MAC operation that produces the intermediate matrix $\mathbf{p}_2$ used in the covariance update:

$$\mathbf{p}_2 = \mathbf{\Phi} - \mathbf{k} \otimes \mathbf{p}^T. \tag{21}$$

The final operation is a scalar division on the $\mathbf{p}_2$ matrix to generate the updated covariance matrix, $\mathbf{\Phi}_{\text{new}}$. The scalar λ is a pre-determined constant during the RLS training process, typically within the range of $(0.9, 1)$. The equation is:

$$\mathbf{\Phi}_{\text{new}} = \frac{\mathbf{p}_2}{\lambda}. \tag{22}$$

Since λ is fixed, we can hard-code it and simplify the division operation in a more precise manner than the division used in the *weight_update* module. The division can be approximated into a shift & add operation:

$$\mathbf{\Phi}_{\text{new}} = \mathbf{p}_2 + \frac{\mathbf{p}_2}{2^m}, \tag{23}$$

where m is determined based on the specific value of λ. In our implementation, with $\lambda = 0.999$, the parameter m is set to 10, which introduces a negligible approximation error of the order of 10^{-5}.

3) Pipeline Design: Given that these operations involve high-dimensional inputs, we leverage the SIMD (Single Instruction, Multiple Data) characteristics of Equation 15, 18 and Equation 21, 23 by implementing a pipeline design with resource sharing. For the two MVM operations in equation 12 and 20, we implemented two DSP_SA-based matrix-vector multipliers to enable parallel computation and mitigate high latency. And the equation 13 and 14 need to be completed before the pipeline due to the data dependency of u_3.

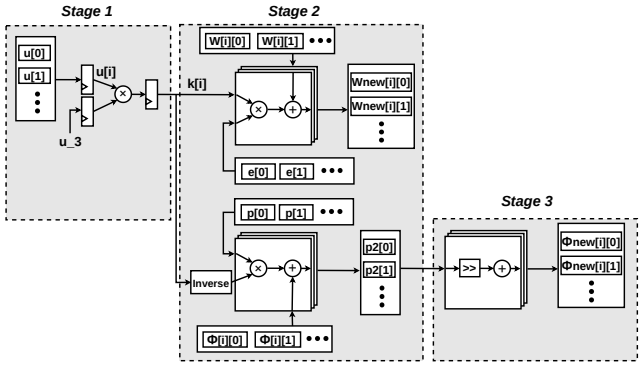


Fig. 5. The pipeline design in RLS training

Figure 5 displays the proposed 3-stage pipeline to support batch processing for operations.

In stage 1, a single multiplier is used to generate one element k_i of the gain vector, in every two cycles.

In stage 2, two sets of MAC operations, denoted as MAC1 and MAC2, are performed in parallel, corresponding to Equation 18 and 21. Equation 18 can be decomposed into element-wise MAC operations:

$$W_{\text{new},ij} = e_i \cdot k_j + W_{ij}, \quad \forall i \in [0, N_o - 1],\ j \in [0, N_i + N_r - 1]. \tag{24}$$

Similarly, Equation 21 is expressed as:

$$p_{2,ij} = p_i \cdot (-k_j) + \Phi_{ij}, \quad \forall i \in [0, N_i + N_r - 1],\ j \in [0, N_i + N_r - 1]. \tag{25}$$

For each MAC group, $(N_i + N_r)$ DSPs are allocated to compute all i-indexed MAC operations for a fixed j index in parallel. The resulting partial $\mathbf{W}_{\text{new}}$ is registered and written back to the output weight memory, while the partial $\mathbf{p}_2$ is registered for use in the next stage.

In stage 3, the shift-and-add operation for Equation 23 is performed. Since the shift bit length is pre-determined and the variables are designed in fixed-point format (as discussed in the next section), the shift operation can be replaced by a simple indexing and sign extension. Using the partial $\mathbf{p}_2$ from the second stage, the partial $\mathbf{\Phi}_{\text{new}}$ can be generated and written back to the $\mathbf{\Phi}$ memories.

With the proposed pipeline, it takes $2 \times (N_i + N_r) + 2 + 2$ clock cycles to update both the $\mathbf{W}_{\text{out}}$ matrix and the $\mathbf{\Phi}$ matrix.

D. System-level fixed-point design

In this system, we adopt a Q-format fixed-point design to balance precision and cost efficiency. A number in Q$m.n$ format consists of one sign bit, m integer bits, and n fractional bits, representing the real value as $x_Q = x \times 2^{-n}$, where x is the signed integer stored in the fixed-point register.

TABLE I
THE Q-FORMAT DESIGN FOR ONLINE ESN SYSTEM

Parameter in Reservoir Layer	***Range***	***Q-Format Design***
Scaled Inputs	(-0.3, 0.3)	Q0.15
Input weights	(-1, 1)	Q0.15
Reservoir weights	(-0.5, 0.5)	Q0.15
Reservoir states	(-1, 1)	Q0.19
a (in Tanh approximation)	8	-
Depth of Tanh LUT	2^8	-
Parameter in Output Layer	***Range***	***Q-Format Design***
Output weights	(-0.3, 0.3)	Q0.15
Outputs	(-1.5, 1.5)	Q2.19
Prediction error e	(-1.5, 1.5)	Q2.19
Parameter in RLS Learning	***Range***	***Q-Format Design***
Covariance matrix Φ	(-2, 2)	Q2.14
u	(-1, 1)	Q0.17
u_2	(2, 16)	Q5.12
Kalman gain k	(-0.15, 0.15)	Q0.17
p	(-1, 1)	Q0.16
p_2	(-2, 2)	Q2.14

We first perform floating-point simulations to determine each variable's range, which sets the number of integer bits, and then iteratively tune the fractional bits until the target precision is met. The resulting Q-format for all variables is summarized in Table I.

E. Overall Latency and Resource Tradeoff

The three compute modules of the proposed system all exhibit latencies *linear* in the extended-state dimension $L = N_i + N_r = N_o + N_r$: the reservoir update takes $\lceil L/9 \rceil + 2$ cycles, each DSP_SA-based matrix–vector multiplication takes $N + L + 1$ cycles (where N is the fixed DSP_SA array size, a small hardware constant independent of the problem size), and the pipelined RLS update takes $2L+4$ cycles. Since the parallel DSP_SA units and L-wide MAC groups make these latencies independent of N_o, both the inference and per-sample training latency scale as $\mathcal{O}(L)$. This is notable given that the arithmetic workload is quadratic—each RLS sample needs $\mathcal{O}(L^2 + N_o L)$ operations (Section II-B), dominated by the covariance update. The architecture maps this quadratic complexity onto a linear latency through spatial parallelism, i.e., at the cost of a linearly growing amount of arithmetic and memory resources, which is consistent with the near-linear resource scaling in Figure 8.

V. EXPERIMENTAL EVALUATION

This section evaluates the proposed on-chip ESN architecture from both hardware-efficiency and application-level prediction perspectives. The experimental evaluation is organized into three parts. First, we assess the proposed MAC_DA and DSP_SA units using a MVM workload, which is one of the dominant operations in the on-chip ESN design. This experiment is used to quantify the hardware performance of the proposed blocks in comparison with representative FPGA-based baseline designs. Second, we evaluate the complete on-chip ESN system using the Mackey–Glass time-series prediction task. This benchmark is widely used for validating nonlinear temporal modeling capability and allows us to compare the proposed hardware implementation with previously reported reservoir-computing and ESN hardware platforms. The comparison is conducted using normalized mean squared error (NMSE), and both software-level and hardware-level results are reported. Third, we evaluate the proposed ESN in a practical real-time wireless channel prediction task for a MIMO-OFDM system. This experiment is designed to demonstrate the applicability of the proposed architecture to communication scenarios where outdated channel state information (CSI) can degrade transmission performance. Realistic channel traces are generated using NS-3 with a 3GPP spatial channel model, and the ESN is trained to predict future channel states from historical channel observations.

A. MAC Unit Comparison with SoTA

As two of the key modules, we compare the synthesis results and performance of the proposed MAC_DA unit and DSP_SA unit with the baseline MAC architectures in [22], [23], and [9]. For the proposed MAC_DA and DSP_SA units, as well as the baseline design in [9], a matrix–vector multiplication between a 72×72 matrix and a 72×1 vector is used to enable a relatively fair comparison. The remaining baseline architectures are evaluated using their respective matrix dimensions as reported in the original works. The comparison results are summarized in Table II.

It can be observed that the MAC_DA- and DSP_SA-based designs achieve the top 2 throughputs among all designs. In terms of normalized resource cost (NRC), the MAC_DA is at a middle level among the baselines, while the DSP_SA design achieves a pretty low cost, only worse than the baseline in [22]. A more fair comparison in area efficiency further proves that

TABLE II
COMPARISON OF DIFFERENT MAC BLOCK DESIGNS FOR MATRIX MULTIPLICATION

		[22], 2018	[23], 2021	[9], 2021	MAC_DA	DSP_SA
Device		Xilinx Virtex-4	Intel Arria10	Xilinx Virtex-7	Xilinx Virtex-7	Xilinx Virtex-7
Function		MV MULT	MV MULT	MV MULT	MV MULT	MV MULT
Matrix Size		1024×28, 28×1	4×1024, 1024×1	72×72, 72×1	72×72, 72×1	72×72, 72×1
Frequency (MHz)		50	259	100	100	100
Data Type		INT	INT8	Fixed-point 16-bit	Fixed-point 16-bit	Fixed-point 16-bit
Resource Utilization	LUT	3200	186534	9176	25016	3127
	FF (register)	7511	-	328	13688	1711
	DSP	21	1176	72	80	72
	DSP Efficiency (MOPS/DSP)	-	-	65.0	81.1	88.9
	BRAM	0	1178	36	36	36
NRC[1]		16591	702348	29664	61104	24998
Computation Time		17.76μs	-	1.04μs	0.9μs	0.81μs
Throughput (GOPS)[2]		3.17	-	9.9	11.44	12.71
Area Efficiency (GOPS/kNRC)[3]		0.191	-	0.334	0.187	0.508
Power (W)		-	-	0.183	0.253	0.246

1. The normalized resource consumption (NRC) is defined as *NRC=LUT+FF+DSP×280*. If the number of FF is not reported, an approximate expression is adopted: *NRC=LUT×2+DSP×280*.
2. Throughput is defined as the number of operations per second. Both MULT and ADD are defined as one operation, MAC is defined as two operations.
3. The area efficiency of FPGA implementation is defined as *Throughput / kNRC*.

the DSP_SA design has the highest area efficiency, achieving 2.66× improvement over [22], and 1.52× improvement over [9].

Also, both proposed designs exhibit higher DSP efficiency than the baseline in [9], achieving a better balance between DSP and CLB utilization. This balanced resource usage leads to slightly increased power consumption; however, such trade-offs are acceptable given that the primary design objective is to optimize the cost efficiency of the complete on-chip ESN system.

B. Mackey-Glass Task

The Mackey-Glass system is a nonlinear differential equation widely used in studies of chaotic dynamics and time-series prediction. By adjusting its parameters, the system can exhibit a range of behaviors, from periodic to chaotic. The system is defined as

$$\hat{y}(t) = \frac{\alpha\, y(t-\tau)}{1 + y(t-\tau)^{\beta}} - \gamma\, y(t), \tag{26}$$

where $y(t)$ is the system state at time t, α is the production gain controlling the delayed feedback, β is the nonlinearity exponent shaping the saturating response, γ is the linear decay rate, and τ is the feedback delay. For our task we set $\alpha = 0.2$, $\beta = 10$, $\gamma = 0.1$, and $\tau = 17$ to create a mildly chaotic attractor, suitable for testing the predictive capability of the ESN.

1) Experiment results: We evaluate the ESN implementation in both software and hardware, using normalized mean square error (NMSE) as the performance metric. The NMSE can be defined as:

$$\text{NMSE}(y, \hat{y}) = \frac{\sum_{n=1}^{N} \left(\hat{y}(n) - y(n)\right)^2}{\sum_{n=1}^{N} \left(y(n) - \bar{y}(n)\right)^2} \tag{27}$$

where $\hat{y}(n)$ is the predicted value, $y(n)$ is the ground-truth value, $\bar{y}(n)$ is the mean of the ground-truth values, and N is the number of time stamps.

We compare the NMSE of the proposed architecture with previously published hardware implementations. The results, as shown in Table III, demonstrate that our design effectively models the Mackey-Glass system, achieving performance comparable to the average levels in the literature and nearly matching the accuracy of the Python-based implementation.

C. MIMO-OFDM Channel Prediction Task

The performance of the proposed on-chip ESN is also evaluated for a wireless channel prediction task in a MIMO-OFDM scenario, in which accurate channel information at the transmitter is essential. In time-division duplex systems, channel reciprocity can be exploited to avoid explicit feedback; however, under high-mobility conditions, processing delays still cause the channel observed at the transmitter to become inaccurate. In frequency-division duplex systems, the receiver estimates the channel and reports it back to the transmitter, but the feedback delay causes the CSI to be outdated by the time it is used. This degradation of CSI accuracy over time is referred to as the **channel aging** effect.

Channel prediction addresses channel aging by estimating the current channel from historical, outdated CSI, thereby improving the CSI quality without consuming additional wireless resources. Existing prediction algorithms generally fall into three categories: parametric methods, autoregressive (AR) models, and machine learning (ML)-based methods.

AR and parametric models have traditionally been used for high-mobility scenarios due to their lower complexity, but ML-based techniques have been shown to outperform them [27]. In this work, we evaluate our on-chip ESN for channel prediction in the mobile MIMO system, modeled as a time-series forecasting task.

1) Realistic channel generation: For the channel prediction task, channel data have been generated with Network Simulator-3 (NS-3), which utilizes a 3GPP Spatial channel

TABLE III
COMPARISON WITH PREVIOUSLY PUBLISHED RESULTS FOR THE MACKEY–GLASS SERIES

Technology	N	NMSE (Hardware)	NMSE (Simulation)	$-\log_{10}$(RMSE)	Integrated Functions
Optoelectronic [24]	900	0.013	-	-	Inference
Cyclone V FPGA [25]	100	1.92×10^{-4}	1.92×10^{-4}	2.63	Inference
Cyclone IV FPGA [26]	300	0.010	-	1.74	Inference
Arria 10 FPGA [19]	128	0.256	0.199	1.06	Inference & Training
Virtex-7 FPGA (Proposed ESN)	128	0.048	0.043	2.34	Inference & Training

model (SCM) to simulate vehicle-to-vehicle (V2V) channel conditions [28]. The settings involve a MIMO system with one transmitter and one receiver, each equipped with two antennas.

The channel generation configuration is detailed in the following table IV:

TABLE IV
CHANNEL GENERATION CONFIGURATION

Parameter	*Value*
Channel scenario	V2V-Urban
Transmit Antenna	2
Receive Antenna	2
Speed	10 km/h
Distance between BS and UE	500 m
Number of subframes	50
Number of subcarriers	8
Number of symbols per subframe	14
Number of pilot symbols per subframe	4

2) Problem formulation: Consider a narrowband MIMO baseband model. The received vector at discrete time index k can be expressed as

$$\mathbf{y}_k = \mathbf{H}_k \mathbf{s}_k + \mathbf{w}_k, \tag{28}$$

where $\mathbf{y}_k \in \mathbb{C}^{N_R \times 1}$ is the received signal across the N_R antennas, $\mathbf{s}_k \in \mathbb{C}^{N_T \times 1}$ is the transmitted symbol vector from the N_T antennas, and $\mathbf{w}_k \in \mathbb{C}^{N_R \times 1}$ denotes additive white Gaussian noise. The MIMO channel is represented by the matrix $\mathbf{H}_k \in \mathbb{C}^{N_R \times N_T}$ whose (i,j)-th entry, $[\mathbf{H}_k]_{i,j}$, captures the complex gain from transmit antenna j to receive antenna i at time k.

In practice, the CSI available at the transmitter is delayed due to estimation, processing, and feedback latency. Let Δ denote this delay; the CSI used for transmission at time $k+\Delta$ is then based on an earlier observation, and since the channel has generally evolved, $\mathbf{H}_k \neq \mathbf{H}_{k+\Delta}$.

Given a window of past channel measurements $\mathcal{S}_k = \{\mathbf{H}_{k-n+1}, \mathbf{H}_{k-n+2}, \ldots, \mathbf{H}_k\}$, channel prediction learns a mapping $\widehat{\mathbf{H}}_{k+\Delta} = \Psi(\mathcal{S}_k)$ from observations up to time k. The model $\Psi(\cdot)$ is trained to minimize the mean-squared prediction error $\min_\Psi \|\mathbf{H}_{k+\Delta} - \widehat{\mathbf{H}}_{k+\Delta}\|_2^2$, and its performance is evaluated using $\mathrm{NMSE}(\mathbf{H}, \widehat{\mathbf{H}})$.

D. Baseline Autoregressive Channel Prediction Model

Wireless channels exhibit strong temporal correlation across consecutive time instants, particularly under moderate mobility. A common way to exploit this structure is to model the channel as a finite-order autoregressive (AR) process, expressing the current realization as a weighted combination of its past values—a tractable linear model for short-term extrapolation. Prior studies estimate the AR coefficients within state-space formulations, e.g., using Kalman filtering for recursive prediction from noisy observations [29], and AR predictors have been shown to perform well for narrowband single-input single-output (SISO) channels using only a few past samples [30].

For MIMO systems, AR prediction is commonly applied by treating each transmit–receive antenna pair as an independent SISO link. While this decomposition reduces computational complexity, it ignores the spatial correlation across antenna elements and therefore cannot fully exploit the multi-antenna structure. An AR-based channel predictor of order p can be written as

$$\widehat{\mathbf{H}}[t] = \sum_{i=1}^{p} \alpha_i \, \widehat{\mathbf{H}}[t-i], \tag{29}$$

where $\widehat{\mathbf{H}}[t]$ is the predicted channel at time index t, $\{\alpha_i\}_{i=1}^{p}$ are the AR coefficients learned from historical observations, and p specifies the number of past channel instances used in the prediction. Its accuracy depends critically on the validity of the linear temporal assumption and the chosen p.

E. Echo State Network–Based Channel Prediction

ESN-based channel prediction is a learning-driven technique that does not require an explicit model to describe the relationship between past and future channel states. Instead of assuming a predefined temporal structure, the ESN learns the relationship between past and future channel realizations directly from observed data. This makes the approach particularly attractive in scenarios where channel dynamics are complex or difficult to model accurately using conventional analytical methods.

In the proposed framework, the dimensionality of the ESN input and output vectors is determined by the number of transmit antennas, receive antennas, and frequency-domain subcarriers. Including subcarrier-wise channel coefficients as features allows the model to capture frequency selectivity and temporal variation jointly. Since the ESN operates on real-valued inputs, the complex-valued channel coefficients are decomposed into their real and imaginary components, which are concatenated to form the final feature representation. Consequently, the input dimension and output dimension is

$$N_{\text{feature}} = T_{\text{ant}} \times R_{\text{ant}} \times N_{\text{sc}} \times 2,$$

where T_{ant} and R_{ant} denote the number of transmit and receive antennas, and N_{sc} is the number of subcarriers. In our generated channel dataset, the N_{feature} turns out to be 64.

Channel prediction is performed in a symbol-level forecasting manner. Specifically, the ESN is trained to predict the channel state at a fixed prediction horizon of 20 symbols ahead using the channel observations from the current subframe. Let

$\mathbf{D}$ denote the input sequence containing channel coefficients of the present subframe, and let $\mathbf{I}$ represent the corresponding future channel sequence used as the training target. The input–label pairing can be expressed as

$$\Phi(\mathbf{I}, \mathbf{D}) = \left\{ \left[H_{t+20}, H_{t+21}, \dots \right], \left[H_t, H_{t+1}, \dots \right] \right\},$$

where $\{H_t, H_{t+1}, \dots\}$ denotes the channel realizations associated with consecutive symbols in the current subframe, and $\{H_{t+20}, H_{t+21}, \dots\}$ corresponds to the channel states at the chosen prediction offset.

For the 2×2 MIMO configuration in our simulations, the ESN uses 64 input and 64 output neurons, a reservoir of 8 neurons, a sparsity of 0.8, a spectral radius of 0.7, a learning rate of 0.001, and an input scaling of 10.

F. Experiment Results

1) NMSE performance evaluation: To evaluate the proposed on-chip ESN for channel prediction, we compare against one non-ML and three ML-based baselines, along with ESN references in CPU and GPU:

- **AR**: a classical linear autoregressive predictor with order $p = 20$, whose coefficients are estimated via the Yule–Walker equations.
- **MLP**: a fully connected feed-forward network with no explicit temporal structure, serving as a non-recurrent reference.
- **CNN**: a one-dimensional convolutional network that extracts local temporal patterns from the input window.
- **GRU**: a gated recurrent network that captures longer-range temporal dependencies via reset and update gates.
- **ESN (CPU/GPU)**: floating-point references of the same ESN architecture implemented in CPU and GPU.

The MLP, CNN, and GRU models are independently and fully tuned for each SNR scenario, so their NMSE figures represent an upper bound on the accuracy each model can achieve. In our cases, both the transmitter and receiver are assumed to move at 10 km/h.

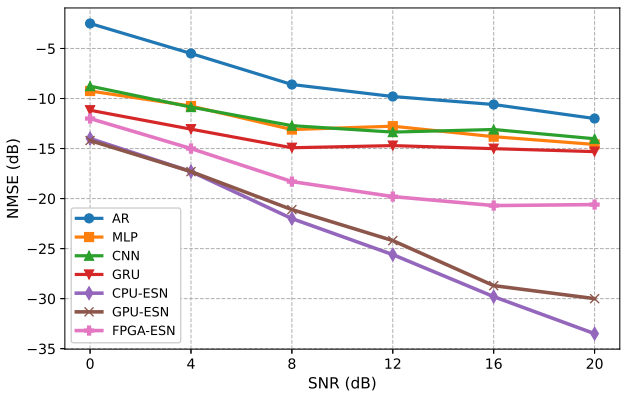


Fig. 6. NMSE Vs SNR performance of different channel prediction models.

Figure 6 compares the NMSE performance of different channel prediction methods across varying SNR levels. Overall, ESN-based predictors consistently outperform both the non-ML AR method and other ML-based models (MLP, CNN, and GRU) over the entire SNR range, demonstrating the strong temporal modeling capability of ESNs for dynamic wireless channels.

Among the ESN implementations, the Python-based ESN achieves the lowest NMSE, followed closely by the GPU-based ESN, while the proposed FPGA-based ESN exhibits slightly higher error. The performance differences between the FPGA- and CPU/GPU-based ESN predictor are minor at low SNR levels, with an NMSE gap of approximately 2.0–2.3 dB at 0–4 dB SNR, where channel noise dominates the overall error. As the SNR increases, the gap between the FPGA-based ESN and its floating-point counterparts becomes more pronounced, increasing to about 5.8 dB at 12 dB SNR and further to 12.9 dB at 20 dB SNR. This behavior can be attributed to fixed-point quantization, rounding, and hardware approximations in the FPGA implementation.

Nevertheless, despite these hardware-induced accuracy losses, the FPGA-based ESN maintains competitive performance and significantly outperforms all non-ESN baselines across the entire SNR range. These results demonstrate the robustness of the ESN model for channel prediction and the reliable and effective realization of ESN on FPGA platforms under practical fixed-point constraints.

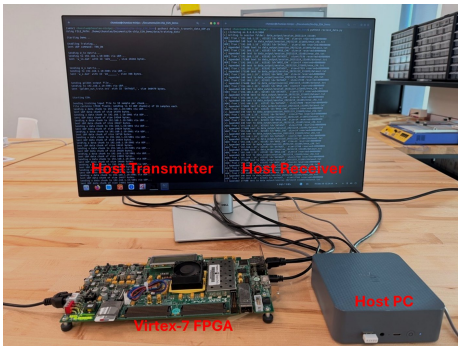


Fig. 7. Real-time testbed for the proposed on-chip ESN

2) FPGA Synthesis Results: The proposed design is further validated using a real-time testbed, as illustrated in Figure 7. The dataset is stored on a host PC, while the on-chip ESN is deployed on a Virtex-7 FPGA board. Data exchange between the host and the FPGA is carried out via an Ethernet interface using the UDP protocol. In addition, Python-based transmitter and receiver modules are developed on the host side for real-time interaction.

In the testing, we performed real-time operations at a global clock of 100 MHz. The latency of the optimized ESN inference design is 0.97 μs, achieving a processing speed of 65.98M input/output samples/s. Each training cycle (including inference) in the channel prediction task achieved a latency of 3.87 μs, supporting large MVM operations with a 72×72 matrix input and a 72×1 vector input.

Table V summarizes the synthesis results of the proposed design and provides a detailed comparison with existing on-chip ESN implementations reported in the literature. It can be observed that the proposed accelerator achieves high through-

put when processing high-dimensional data while maintaining low power consumption.

Since the learning method, ESN size, and data representation differ across designs, area efficiency (AE) and energy efficiency (EE) are evaluated to enable a fair comparison. The proposed design achieves an AE that is $17\times$ higher than that of [19], primarily due to its fully pipelined architecture and efficient reuse of DSP resources during both inference and training. Although the AE is $2.6\times$ lower than that of [17], the latter targets a significantly smaller ESN configuration and processes the 1-bit binary data, whereas the proposed design supports large-scale ESNs with a fixed-point system design. In terms of energy efficiency, the proposed design outperforms [19] by more than $1700\times$, which can be attributed to the use of cost-efficient hardware design strategies and optimized support for high-dimensional data computations, along with the different selection of the training methods.

TABLE V
COMPARISON OF DIFFERENT FPGA ACCELERATORS OF ON-CHIP ESN

		[17], 2016	[19], 2022	Proposed Design
Device		Altera Cyclone-V	Intel Arria-10	Xilinx Virtex-7
Learning Method		Ridge Regression	Pseudo-Inverse	RLS
ESN Size (N_i,N_r,N_o)		(1, 16, 1)	(1, 512, 1)	(64, 8, 64)
Frequency (MHz)		125	125	100
Data Type		1-bit Binary	Fixed-point 8-bit	Fixed-point 16-bit
Resource Utilization	LUT	1060	12305	23153
	FF (register)	2277	8192	8362
	DSP	13	0	433
	BRAM	2.03Mb	0	3.52Mb
NRC		6977	20497	152755
Computation time[4]		0.032μs	4.1μs	3.87μs
Throughput(Mbps)[5]		31.3	1.95	264
AE(Mbps/kNRC)[6]		4.49	0.10	1.73
Power (W)		-	16.22	1.234
EE (Mb/J)[7]		-	0.12	213.94

4. Computation time refers to the total latency required to complete one full training–inference task.
5. Throughput is measured in predicted bits per second (bps), defined as the product of the number of complete training-inference tasks executed per second and the total bits of each prediction output.
6. AE (area efficiency) denotes the number of predicted bits per second per unit of hardware resource and is calculated as *Throughput / NRC*.
7. EE (energy efficiency) denotes the number of predicted bits per joule and is calculated as *Throughput / Power*.

3) Performance Analysis: Based on the results in Section V-F2, we can get the processing time of an entire subframe, which consists of 4 pilot symbols for training and 10 data symbols for testing. The total subframe processing latency is

$$T_{subframe} = T_{train} \times 4 + T_{inference} \times 10 = 0.025ms \quad (30)$$

which satisfies the sub-millisecond latency requirement of Next-G wireless systems.

TABLE VI
SYNTHESIS RESULTS OF THE ON-CHIP ESN FOR THREE OPTIMIZATION STRATEGIES

Optimization	LUT	FF	DSP	BRAM	Processing Time
Balance	23153	8362	433	110	3.87μs
throughput	68413	8382	937	110	3.64μs
Resource	22742	8306	257	110	10.50μs

Table VII compares the throughput and power of the proposed FPGA accelerator with CPU- and GPU-based implementations. For power measurement, FPGA power is taken from the Xilinx Vivado post-implementation power report, CPU power from the Intel RAPL energy counters (pyRAPL), and GPU power via the NVIDIA Management Library (NVML, pynvml). Both baselines implement the identical ESN architecture with the same RLS online training as the proposed design. More specifically, the CPU baseline uses vectorized NumPy linear algebra throughout, while the GPU baseline keeps all reservoir weights, states, and RLS covariance matrices as device-resident PyTorch tensors and runs under `@torch.no_grad()` to minimize host-device transfers and autograd overhead. While CPUs and GPUs provide high computational capability, they incur higher power consumption and less efficient data movement. In contrast, the FPGA-based design leverages hardware parallelism and pipelining to achieve competitive throughput with significantly reduced power consumption. As a result, the energy efficiency of FPGA design is more than $1600\times$ higher than the CPU and $290\times$ higher than the GPU. These results highlight the effectiveness of the FPGA's application-specific parallelism and pipelined architecture for energy-efficient, high-throughput ESN processing in the channel prediction task.

TABLE VII
THROUGHPUT AND POWER COMPARISON BETWEEN OUR ACCELERATOR AND CPU/GPU BASELINES

	Throughput (Mb/s)	Power (W)	EE (Mb/s/W)
CPU(Intel Xeon W-2225)	10.10	75.21	0.13
GPU(NVIDIA RTX A2000)	16.38	22.29	0.73
FPGA (Xilinx Virtex-7)	264	1.234	213.94

This energy-efficiency gap can be attributed to the nature of the workload: the on-chip ESN requires sample-by-sample online adaptation, with matrix-vector multiplications forming the dominant compute load within each sample. This combination leaves limited room for either the CPU's deep control pipeline or the GPU's batch-oriented parallelism to be effectively utilized. The FPGA instead delivers real-time processing through operation-level parallelism, thereby achieving substantially better performance on this workload.

To further analyze the tradeoffs between resource utilization and performance, we explore multiple implementation strategies targeting different optimization objectives, as summarized in Table VI.

For throughput-oriented optimization, the MAC_DA unit increases the number of DSPs from 9 to 72 to complete all multiplications within a single cycle, at the cost of a deeper adder tree, thereby reducing the per-neuron latency from $(8+2)$ to $(1+5)$ clock cycles. In the output layer, the DSP_SA configuration reduces the number of DSPs per array from 8 to 4, which decreases the overall output-layer latency from $(8+72+1)$ to $(4+72+1)$ clock cycles. In the RLS block, further unrolling is constrained by BRAM read–write dependencies, making throughput improvements limited.

For resource-oriented optimization, the MAC_DA unit reduces the DSP count to 3 with simplified accumulation logic. In the DSP_SA unit, the total number of DSPs is kept as 72, since varying the number of DSPs per array does not affect the total number of DSPs. In addition, DSP usage in the RLS block is substantially reduced by lowering the number of

parallel MAC operations from 72 to 8 per cycle, for MAC1 and MAC2.

The balanced strategy corresponds to the architecture proposed in Section IV, with identical numerical precision maintained across all three strategies. The results demonstrate that the proposed balanced architecture achieves an effective tradeoff between throughput and resource utilization.

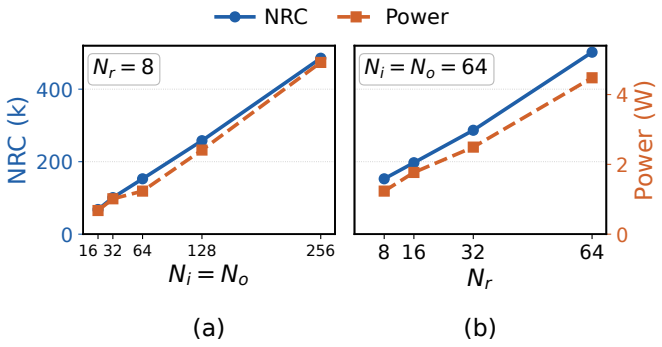


Fig. 8. Scalability of the proposed on-chip ESN: NRC and total power versus (a) the input/output dimension $N_i = N_o$ with N_r fixed at 8, and (b) the reservoir size N_r with $N_i = N_o$ fixed at 64.

Moreover, synthesis results of the proposed on-chip ESN under different ESN configurations are also presented and compared in Figure 8. Two sweeps are conducted: Figure 8(a) varies the input/output dimension $N_i = N_o$ from 16 up to 256 with the reservoir size fixed at $N_r = 8$, while Figure 8(b) varies N_r from 8 up to 64 with $N_i = N_o$ fixed at 64. In both sweeps, the NRC and total power follow a clearly linear trend across the full range of tested configurations.

This near-linear scaling can be explained from two levels. At the architectural level, the hardware parallelism of the design trades the quadratic arithmetic complexity of the RLS update for a linear processing latency, at the cost of a linearly growing amount of compute and memory resources. At the implementation level, the three compute modules are all built on DSP arrays whose DSP counts grow linearly with N_i, N_r, and N_o, and since DSPs dominate the NRC budget, the overall NRC inherits this linear trend.

The total power follows the same trend with a slight upward curvature for larger configurations. This is mainly attributable to the increasing interconnect capacitance from longer and denser routing in larger designs, together with a mild rise in the average switching activity factor as more DSP and BRAM blocks become simultaneously active per cycle.

VI. Conclusion

This paper presents an FPGA-based ESN system with integrated on-chip RLS training for real-time MIMO channel prediction. To support high-speed processing of high-dimensional data, an optimized MAC_DA vector-multiplication unit and a systolic-array-based MVM block (DSP_SA) are proposed for efficient inference, and an optimized on-chip RLS training architecture is developed. Experimental results show that the design effectively supports large-scale channel prediction, outperforming non-ML and ML baselines in accuracy while delivering substantially higher energy efficiency than CPU- and GPU-based implementations. Synthesis results across different ESN configurations confirm its scalability for high-dimensional data, validating the proposed on-chip ESN as a high-speed, energy-efficient solution for real-time channel prediction in Next-G wireless systems.

Acknowledgment

The authors used Claude for language polishing of the author-written text in this manuscript. All technical content, methodology, experimental results, figures, and code were developed and verified by the authors.

References

[1] Z. Zhou, M. Shojafar, J. Abawajy, H. Yin, and H. Lu, "Ecms: An edge intelligent energy efficient model in mobile edge computing," *IEEE Transactions on Green Communications and Networking*, vol. 6, no. 1, pp. 238–247, 2021.

[2] Z. Zhou and J. Abawajy, "Reinforcement learning-based edge server placement in the intelligent internet of vehicles environment," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[3] U. S. Khan, L. Liu, S. Jere, L. Zheng, and Y. Yi, "Configuring rnn weights for mimo-ofdm receive processing: Informing rnn with domain knowledge," *IEEE Wireless Comm. Letters*, pp. 1–1, 2025.

[4] H. Jaeger, "Echo state network," *scholarpedia*, vol. 2, no. 9, p. 2330, 2007.

[5] J. Xu, S. Jere, Y. Song, Y.-H. Kao, L. Zheng, and L. Liu, "Learning at the speed of wireless: Online real-time learning for ai-enabled mimo in nextg," *IEEE Communications Magazine*, 2024.

[6] J. Xu, Z. Zhou, L. Li, L. Zheng, and L. Liu, "Rc-struct: A structure-based neural network approach for mimo-ofdm detection," *IEEE Transactions on Wireless Communications*, vol. 21, no. 9, pp. 7181–7193, 2022.

[7] K. A. Said, Y. Liang, U. Saeed, R. Safavinejad, K. S. Bondada, E. Allen, N. Mohammadi, B. Pimentel, B. Kelley, J. Reed *et al.*, "Mobile distributed mimo (md-mimo) for nextg: Mobility meets cooperation in distributed arrays," *arXiv preprint arXiv:2504.12244*, 2025.

[8] A. Rodan and P. Tino, "Minimum complexity echo state network," *IEEE transactions on neural networks*, vol. 22, no. 1, pp. 131–144, 2010.

[9] V. M. Gan, Y. Liang, L. Li, L. Liu, and Y. Yi, "A cost-efficient digital esn architecture on fpga for ofdm symbol detection," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 17, no. 4, pp. 1–15, 2021.

[10] C. Lin, Y. Liang, and Y. Yi, "Fpga-based reservoir computing with optimized reservoir node architecture," in *2022 23rd International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2022, pp. 1–6.

[11] C. Lin, M. F. Azmine, and Y. Yi, "Accelerating next-g wireless communications with fpga-based ai accelerators," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.

[12] C. Lin, M. F. Azmine, Y. Liang, and Y. Yi, "Leveraging neuro-inspired ai accelerator for high-speed computing in 6g networks," *Frontiers in Computational Neuroscience*, vol. 18, p. 1345644, 2024.

[13] Y. Liang, L. Li, Y. Yi, and L. Liu, "Real-time machine learning for symbol detection in mimo-ofdm systems," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 2068–2077.

[14] K. Caluwaerts, M. D'Haene, D. Verstraeten, and B. Schrauwen, "Locomotion without a brain: physical reservoir computing in tensegrity structures," *Artificial life*, vol. 19, no. 1, pp. 35–66, 2013.

[15] R. Legenstein, S. M. Chase, A. B. Schwartz, and W. Maass, "A reward-modulated hebbian learning rule can explain experimentally observed network reorganization in a brain control task," *Journal of Neuroscience*, vol. 30, no. 25, pp. 8400–8410, 2010.

[16] T. Kailath, A. H. Sayed, and B. Hassibi, *Linear estimation*. Prentice Hall, 2000.

[17] Y. Yi, Y. Liao, B. Wang, X. Fu, F. Shen, H. Hou, and L. Liu, "Fpga based spike-time dependent encoder and reservoir design in neuromorphic computing processors," *Microprocessors and Microsystems*, vol. 46, pp. 175–183, 2016.

[18] P. Antonik, A. Smerieri, F. Duport, M. Haelterman, and S. Massar, "Fpga implementation of reservoir computing with online learning," in *24th Belgian-Dutch Conference on Machine Learning*, 2015, p. 99.

[19] F. Galán-Prado and J. L. Rosselló, "A noniterative supervised on-chip training circuitry for reservoir computing systems," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 3, pp. 4097–4109, 2022.
[20] T. Luo, P. Chen, Z. Wang, and Y. Zhang, "Recursive least square anti-jamming algorithm and fpga implementation for beidou navigation," in *2022 IEEE 5th International Conference on Electronics Technology (ICET)*. IEEE, 2022, pp. 874–878.
[21] H. H. Thannoon and I. A. Hashim, "Efficient fpga implementation of recursive least square adaptive filter using non-restoring division algorithm," *International Journal of Electronics and Telecommunications*, pp. 811–817, 2023.
[22] I. Sayahi, M. Machhout, and R. Tourki, "Fpga implementation of matrix-vector multiplication using xilinx system generator," in *2018 International Conference on Advanced Systems and Electric Technologies (IC_ASET)*. IEEE, 2018, pp. 290–295.
[23] Z. Que, H. Nakahara, E. Nurvitadhi, A. Boutros, H. Fan, C. Zeng, J. Meng, K. H. Tsoi, X. Niu, and W. Luk, "Recurrent neural networks with column-wise matrix–vector multiplication on fpgas," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 2, pp. 227–237, 2021.
[24] J. Bueno, S. Maktoobi, L. Froehly, I. Fischer, M. Jacquot, L. Larger, and D. Brunner, "Reinforcement learning in a large-scale photonic recurrent neural network," *Optica*, vol. 5, no. 6, pp. 756–760, 2018.
[25] F. Galán-Prado, J. Font-Rosselló, and J. L. Rosselló, "Tropical reservoir computing hardware," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 11, pp. 2712–2716, 2020.
[26] M. L. Alomar, E. S. Skibinsky-Gitlin, C. F. Frasser, V. Canals, E. Isern, M. Roca, and J. L. Rosselló, "Efficient parallel implementation of reservoir computing systems," *Neural Computing and Applications*, vol. 32, pp. 2299–2313, 2020.
[27] F. Peng, S. Zhang, Z. Jiang, X. Wang, and W. Chen, "A Novel Mobility Induced Channel Prediction Mechanism for Vehicular Communications," *IEEE Trans. on Wireless Commun.*, vol. 22, no. 5, pp. 3488–3502, 2023.
[28] T. Zugno, M. Polese, N. Patriciello, B. Bojović, S. Lagen, and M. Zorzi, "Implementation of a spatial channel model for ns-3," in *Proceedings of the 2020 Workshop on ns-3*, 2020, pp. 49–56.
[29] H. Kim, S. Kim, H. Lee, C. Jang, Y. Choi, and J. Choi, "Massive mimo channel prediction: Kalman filtering vs. machine learning," *IEEE Transactions on Communications*, vol. 69, no. 1, pp. 518–528, 2020.
[30] W. Jiang and H. D. Schotten, "Neural network-based fading channel prediction: A comprehensive overview," *IEEE Access*, vol. 7, pp. 118 112–118 124, 2019.



Chunxiao Lin received his B.S. degree in electrical engineering from HUAZHONG University of Science and Technology, in 2019, and his M.S. degree in electrical and computer engineering from the University of Southern California in 2021. He is currently pursuing his Ph.D. degree in Bradley Department of Electrical and Computer Engineering at Virginia Tech. His research interests include FPGA system design, reservoir computing, and wireless communication.



Ummay Sumaya Khan received her B.Sc. degree in electrical and electronic engineering from Bangladesh University of Engineering and Technology, in 2017, and her M.S. degree in electrical and computer engineering from the University of Florida in 2023. She is currently pursuing her Ph.D. degree at the Wireless@VT, The Bradley Department of Electrical and Computer Engineering, Virginia Tech. Her research interests include wireless communications, machine learning, neuromorphic computing, information theory, and statistical learning theory.



Meiyu Zheng received her B.S. degree in Computer Engineering from Virginia Tech in 2024 and her M.Eng. degree in Computer Engineering from Virginia Tech in 2025. Her work focuses on embedded systems and FPGA-based design. Her research interests include neuromorphic computing and machine learning hardware, with an emphasis on bridging algorithms and hardware for real-time applications.



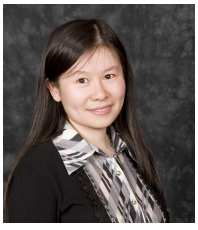
Muhammad Farhan Azmine is a Ph.D. researcher in the BRICCS lab & the MICS group at the Bradley Department of Electrical and Computer Engineering, Virginia Tech. He received his M.S. in Computer Engineering from Virginia Tech (2024) and his B.S. in Electrical & Electronic Engineering from Bangladesh University of Engineering & Technology (2017). His research focuses on neuromorphic computing, spiking neural networks, hardware algorithm co-design, and PPA-aware RTL design for energy-efficient ML accelerators targeting edge AI.



Daniel J. Jakubisin is a research associate professor with the Hume Center for National Security and Technology, National Security Institute and the Bradley Department of Electrical and Computer Engineering (by courtesy), Virginia Tech. He is a member of Wireless@Virginia Tech and his research focus is resilient wireless communication. He has experience with 4G/5G/O-RAN protocols, standards, and IP; future 6G waveform and architecture research.



Lingjia Liu is a professor of Wireless@Virginia Tech, the Bradley Department of Electrical and Computer Engineering, Virginia Tech. He directs Wireless@Virginia Tech, focusing on 5G/6G networks, Machine Learning in Wireless Tech, Open RAN, and Dynamic Spectrum Sharing.



Yang Yi (Senior Member, IEEE) is a professor with the Bradley Department of Electrical and Computer Engineering, Virginia Tech. Her research interests include VLSI circuits and systems, computer-aided design, neuromorphic architecture for braininspired computing systems, and low-power circuits design with advanced nano-technologies for high-speed wireless systems.